COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/24/2026	**DATE SUBMITTED:**	7/29/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

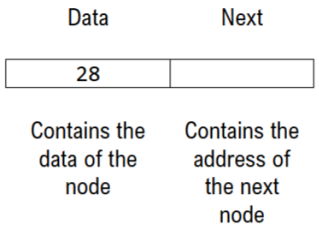
ACTIVITY NO. 3.1 Link Lists

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

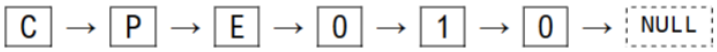
To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list.



Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list. In C++, the node is defined as:

```
class Node{
public:
        char data;
        Node *next;
};
```

We will implement a list to represent the string “CPE010” that looks like the figure below:



Implementation will follow the given steps:
1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null

Simple implementation:

Screenshot

main.cpp

Single Linklist > main.cpp > main()

```cpp
#include <iostream>
#include <utility>

class Node {
public:
    char data;
    Node *next;
};

int main() {
    // Step 1: Declare pointers
    Node *head = nullptr;
    Node *second = nullptr;
    Node *third = nullptr;
    Node *fourth = nullptr;
    Node *fifth = nullptr;
    Node *last = nullptr;

    // Step 2: Allocate memory for nodes
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;

    // Step 3: Assign data and link nodes
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '0';
    fourth->next = fifth;

    fifth->data = '1';
    fifth->next = last;

    // Step 4: Assign data to the last node and terminate the list
    last->data = '0';
    last->next = nullptr;

    // Optional: Clean up dynamically allocated memory
    delete head;
    delete second;
    delete third;
    delete fourth;
    delete fifth;
    delete last;

    return 0;
}
```

OUTPUT:

```
PS C:\Users\Justine\Desktop\Hands-on Activity 3.1 Linked Lists>
```

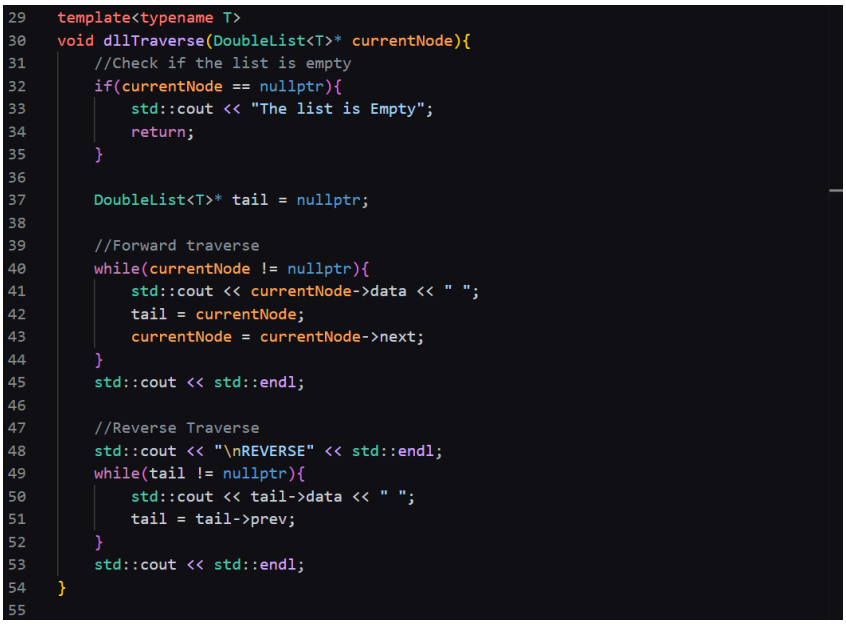
Discussion

After inserting the code in my VS code, the program builds a singly linked list of C → P → E → 0 → 1 → 0 → NULL using six nodes which were dynamically allocated (by the new function). In each node, there is one character in the data member, while the next member contains the pointer to the next node. In the last node, it contains nullptr, indicating the end of the linked list. However, despite the creation of the linked list, the program shows nothing even though there are no errors found in the code.

There are two functions that need to be included in the program to make it more efficient. First, the data of each node needs to be printed to see the output of the linked list on the screen. It is also necessary to free the allocated memory to prevent memory leakage since the program might have deleted and freed the memory but still holds the address (dangling pointers).

This can be avoided by adding a statement such as:
head (name of the node) = nullptr;

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	

```
29  template<typename T>
30  void dllTraverse(DoubleList<T>* currentNode){
31      //Check if the list is empty
32      if(currentNode == nullptr){
33          std::cout << "The list is Empty";
34          return;
35      }
36
37      DoubleList<T>* tail = nullptr;
38
39      //Forward traverse
40      while(currentNode != nullptr){
41          std::cout << currentNode->data << " ";
42          tail = currentNode;
43          currentNode = currentNode->next;
44      }
45      std::cout << std::endl;
46
47      //Reverse Traverse
48      std::cout << "\nREVERSE" << std::endl;
49      while(tail != nullptr){
50          std::cout << tail->data << " ";
51          tail = tail->prev;
52      }
53      std::cout << std::endl;
54  }
55
```

Insertion at head

```cpp
// Inserting a node to the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentNode){

    //Create a new node
    DoubleList<T>* newNode = createNode(newData);

    //Check if the list is empty
    if(*currentNode == nullptr){
        *currentNode = newNode;
        return;
    }

    //Point the newNode to the currentNode (head)
    newNode->next = *currentNode;

    //Point the currentNode(head) back to the newNode
    (*currentNode)->prev = newNode;

    //Update the head pointer
    *currentNode = newNode;
}
```

Insertion at any part of the list

```cpp
//General Insert (Insertion at any part of the list)
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* previousNode){

    //Check if it is the head node (previous node is null)
    if(previousNode == nullptr){
        //If null, print "Previous node cannot be null."
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

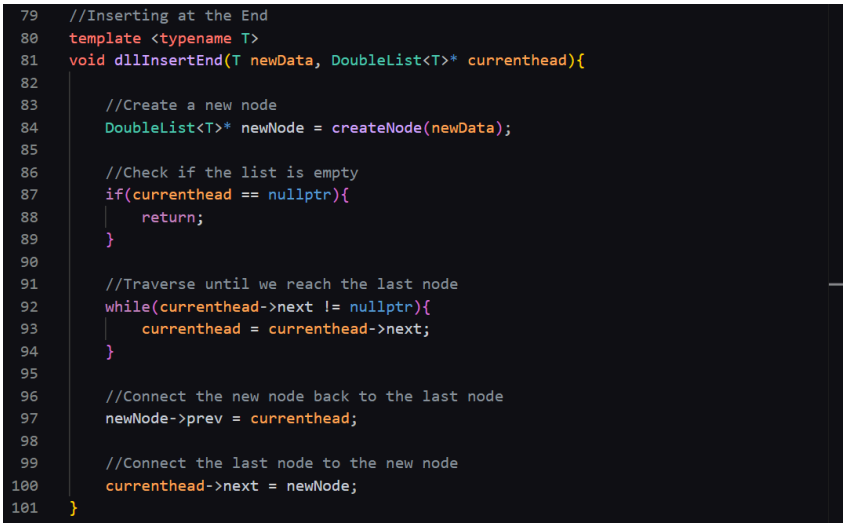
    //Allocate a new node
    DoubleList<T>* newNode = createNode(newData);

    //Store data in the new node
    newNode->data = newData;

    //Point new node to the node previous node was pointing to
    newNode->next = previousNode->next;
    newNode->prev = previousNode;

    //If there is a node after previousNode,
    //connect it back to the new node
    if(previousNode->next != nullptr){
        previousNode->next->prev = newNode;
    }

    //Point previous node to the new node
    previousNode->next = newNode;
}
```

Insertion at the end	
Deletion of a node	

```cpp
//Inserting at the End
template <typename T>
void dllInsertEnd(T newData, DoubleList<T>* currenthead){

    //Create a new node
    DoubleList<T>* newNode = createNode(newData);

    //Check if the list is empty
    if(currenthead == nullptr){
        return;
    }

    //Traverse until we reach the last node
    while(currenthead->next != nullptr){
        currenthead = currenthead->next;
    }

    //Connect the new node back to the last node
    newNode->prev = currenthead;

    //Connect the last node to the new node
    currenthead->next = newNode;
}
```

```cpp
//Deletion of a node
template <typename T>
void dllDeleteNode(DoubleList<T>** head, DoubleList<T>* deleteNode){

    //Check if the list or node is empty
    if(*head == nullptr || deleteNode == nullptr){
        return;
    }

    //If deleting the head node
    if(*head == deleteNode){
        *head = deleteNode->next;
    }

    //Find previous node of the node to be deleted.
    if(deleteNode->prev != nullptr){
        //Change the next of previous node.
        deleteNode->prev->next = deleteNode->next;
    }

    //Update the previous pointer of the next node
    if(deleteNode->next != nullptr){
        deleteNode->next->prev = deleteNode->prev;
    }

    //Free memory for the node to be deleted.
    delete deleteNode;
}

#endif
```

Table 3-2. Code for the List Operations

a.	**Source Code**	`std::cout << "Original List:" << std::endl;` `ListTraversal(head);`

	Console	`Original List:`
		`CPE010`
		Caption:
		Traverses the list by passing the head node to the ListTraversal() function. It prints each node's data in order (CPE010) and stops when it reaches nullptr, which marks the end of the list.

b.	**Source Code**	`// Insert 'G' at the head of the list`
		`std::cout << "Testing sllInsertHead" << std::endl;`
		`sllInsertHead('G', &head);`
		`ListTraversal(head);`

	Console	`Testing sllInsertHead`
		`GCPE010`
		Caption:
		Inserts the element 'G' at the beginning of the list, making it the new head, it is then updated and displayed by traversing the list again.

c.	**Source Code**	`// Insert 'G' at the head of the list`
		`sllInsertHead('G', &head);`
		`// Insert 'E' after the node containing 'P'`
		`std::cout << "Testing sllGeneralInsert" << std::endl;`
		`sllGeneralInsert('E', third);`
		`ListTraversal(head);`

	Console	`Testing sllGeneralInsert`
		`GCPEE010`
		Caption:
		Inserts the element 'E' after the node containing 'P' using the general insert function.

d.	**Source Code**	`// Insert 'G' at the head of the list`
		`sllInsertHead('G', &head);`
		`// Insert 'E' after the node containing 'P'`
		`sllGeneralInsert('E', third);`
		`// Delete the node containing 'C'`
		`std::cout << "Testing sllDeleteNode" << std::endl;`
		`sllDeleteNode('C', &head);`
		`ListTraversal(head);`

	Console	`Testing sllDeleteNode`
		`GPEE010`
		Caption:
		Deletes the node containing the element 'C' and reconnects the remaining nodes.

e.	**Source Code**	`// Insert 'G' at the head of the list`
		`sllInsertHead('G', &head);`
		`// Insert 'E' after the node containing 'P'`
		`sllGeneralInsert('E', third);`
		`// Delete the node containing 'C'`
		`sllDeleteNode('C', &head);`
		`// Delete the node containing 'P'`
		`std::cout << "Testing sllDeleteNode" << std::endl;`
		`sllDeleteNode('P', &head);`
		`ListTraversal(head);`

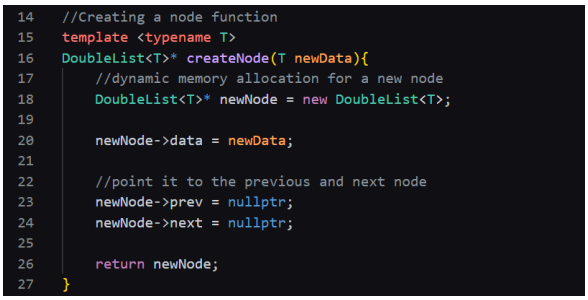
	Console	`Testing sllDeleteNode`
		`GEE010`
		Caption:
		Deletes the node containing the element 'P' and reconnects the remaining nodes.

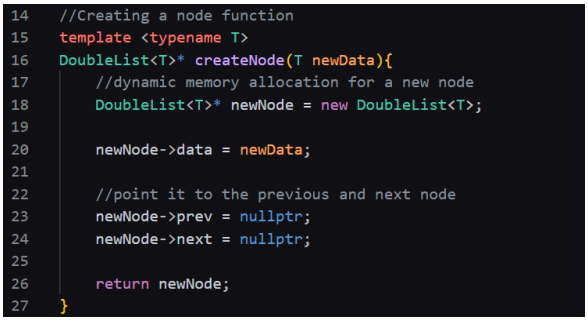
f.	**Source Code**	`std::cout << "New List:" << std::endl;` `ListTraversal(head);`

	Console	`New List:`
		`GEE010`
		Caption:
		Traverses the updated list after the deletion operations and displays the remaining elements in their current order.

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis


``
//Creating a node function
template <typename T>
DoubleList<T>* createNode(T newData){
    //dynamic memory allocation for a new node
    DoubleList<T>* newNode = new DoubleList<T>;
 	
    newNode->data = newData;
 	
    //point it to the previous and next node
    newNode->prev = nullptr;
    newNode->next = nullptr;
 	
    return newNode;
}
``
	In the case of the node structure, a new member variable called prev was added along with the already present next. This way, the node could refer to the previous and the next nodes.


``
//Creating a node function
template <typename T>
DoubleList<T>* createNode(T newData){
    //dynamic memory allocation for a new node
    DoubleList<T>* newNode = new DoubleList<T>;
 	
    newNode->data = newData;
 	
    //point it to the previous and next node
    newNode->prev = nullptr;
    newNode->next = nullptr;
 	
    return newNode;
}
``
	In the singly linked list, only the next pointer was initialized to nullptr. This function was modified to also initialize the prev pointer, ensuring that the new node starts with no previous or next connections before being inserted into the list.


``
template<typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //Check if the list is empty
    if(currentNode == nullptr){
        std::cout << "The list is Empty";
        return;
    }
 	
    DoubleList<T>* tail = nullptr;
 	
    //Forward traverse
    while(currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << std::endl;
 	
    //Reverse Traverse
    std::cout << "\nREVERSE" << std::endl;
    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
``
	The singly linked list traversal used the next pointer for moving forward only. In case of doubly linked list, the traversal was changed such that it first moved forward with keeping track of the last node, then moved backward using prev pointer.

	Inserting at the head only required updating the new node's next pointer and changing the head pointer when it came to the single list. However, in the doubly linked list, the old head's prev pointer must also be updated to point back to the new head, creating a two-way connection, the singly list is only a one way connection.
	In the singly linked list, deleting a node only required updating the previous node's next pointer before freeing the node. In the doubly linked list, both the previous node's next pointer and the next node's prev pointer must be updated.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.

- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

CODE

main.cpp

playlist.h | main.cpp

Playlist > main.cpp > main()

```cpp
#include <iostream>
#include <limits>
#include <string>
#include "playlist.h"

int main(){

    Song* head = nullptr;
    Song* current = nullptr;

    int choice, songNumber;
    std::string songName;

    do{


        std::cout << "\n★.˚★🎧★˚.⋆ Ralph's Playlist Menu⋆.˚★🎧★˚.⋆" << std::endl;
        std::cout << "1. Add Song\n";
        std::cout << "2. Remove Song\n";
        std::cout << "3. Next Song\n";
        std::cout << "4. Play All Songs\n";
        std::cout << "5. Quit\n";

        if(current == nullptr)
            std::cout << "Current Song: None" << std::endl;
        else
            std::cout << "Currently Playing: " << current->title << std::endl;

        std::cout << "\nEnter choice: ";

        while(!(std::cin >> choice)){
            std::cout << "Invalid input. Please enter a number (1-5): ";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }

        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        switch(choice){

        case 1:
            std::cout << "Enter song name: ";
            std::getline(std::cin, songName);

            insertSong(songName, &head);

            if(current == nullptr){
```

```
                current = head;
                std::cout << "\n\nNow Playing: " << current->title << std::endl;
            }

            std::cout << "\nSong has been added to the playlist." << std::endl;
                break;

        case 2:
            if(head == nullptr){
                std::cout << "Playlist is empty." << std::endl;
                break;
            }

            std::cout << "\nCurrent Playlist\n";
            displayPlaylist(head);

            std::cout << "\nEnter song number to remove: ";

            while(!(std::cin >> songNumber)){
                std::cout << "Invalid input. Please enter a number: ";
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            }

            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

            deleteSong(songNumber, &head);

            if(head == nullptr)
                current = nullptr;
            else
                current = head;

            std::cout << "\nSong deleted..." << std::endl;
            break;

        case 3:
        if(current == nullptr){
            std::cout << "Playlist is empty." << std::endl;
        }
        else{
            nextSong(&current);

        }
        break;

        case 4:
```

```
                playPlaylist(head);
                break;

            case 5:
                std::cout << "\n\nEXITING..." << std::endl;
                break;

            default:
                std::cout << "Invalid choice. Please enter a number from 1 to 5." << std::endl;
            }

    }while(choice != 5);

    deletePlaylist(&head);

    return 0;
}
```

playlist.h

playlist.h main.cpp

Playlist > playlist.h > createSong(std::string)

```cpp
#ifndef PLAYLIST_H
#define PLAYLIST_H

#include <iostream>
#include <string>

class Song{
public:
    std::string title;
    Song* next;
};

// Create a new song node
Song* createSong(std::string songTitle){

    // Allocate memory for a new node
    Song* newSong = new Song;
    // Store the song title
    newSong->title = songTitle;
    newSong->next = newSong;

    return newSong;
}

// Add song
void insertSong(std::string songTitle, Song** head){

    // Create a new song
    Song* newSong = createSong(songTitle);
    // Check if the playlist is empty
    if(*head == nullptr){
        *head = newSong;
        return;
    }

    Song* current = *head;
    while(current->next != *head){
        current = current->next;
    }

    // Connect the new song
    current->next = newSong;
    newSong->next = *head;
}

// Display the playlist
void displayPlaylist(Song* head){
```

```cpp

    if(head == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }

    Song* current = head;
    int number = 1;

    do{
        std::cout << number << ". " << current->title << std::endl;
        current = current->next;
        number++;
    }while(current != head);
}

// Play all songs once
void playPlaylist(Song* head){

    if(head == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }

    Song* current = head;
    std::cout << "\nNow Playing:" << std::endl;

    do{
        std::cout << current->title << std::endl;
        current = current->next;
    }while(current != head);
}

// Remove/delete song
void deleteSong(int position, Song** head){

    if(*head == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }

    if(position <= 0){
        std::cout << "Invalid song number." << std::endl;
        return;
    }
```

```
93
94      Song* current = *head;
95      Song* previous = nullptr;
96
97      // Delete the first song
98      if(position == 1){
99
100         if(current->next == *head){
101             delete current;
102             *head = nullptr;
103             return;
104         }
105
106         Song* last = *head;
107
108         while(last->next != *head){
109             last = last->next;
110         }
111
112         *head = current->next;
113         last->next = *head;
114
115         delete current;
116         return;
117     }
118
119     int count = 1;
120
121     previous = current;
122     current = current->next;
123
124     while(current != *head && count < position - 1){
125         previous = current;
126         current = current->next;
127         count++;
128     }
129
130     if(current == *head){
131         std::cout << "Invalid song number." << std::endl;
132         return;
133     }
134
135     previous->next = current->next;
136     delete current;
```

```
137 }
138
139 // Play next song
140 void nextSong(Song** current){
141
142     if(*current == nullptr){
143         std::cout << "Playlist is empty." << std::endl;
144         return;
145     }
146
147     *current = (*current)->next;
148 }
149
150 // Delete the entire playlist and free all the memory
151 void deletePlaylist(Song** head){
152
153     if(*head == nullptr){
154         return;
155     }
156
157     Song* current = (*head)->next;
158
159     while(current != *head){
160         Song* temp = current;
161         current = current->next;
162         delete temp;
163     }
164
165     delete *head;
166     *head = nullptr;
167 }
168
169 #endif
```

OUTPUT:

Menu:

```
★.˚★🎧★˚.★ Ralph's Playlist Menu★.˚★🎧★˚.★
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Current Song: None

Enter choice:
```

Adding a song to the playlist:

```
Enter choice: 1
Enter song name: Understand - Keshi


Now Playing: Understand - Keshi

Song has been added to the playlist.

★.˚★🎧★˚.★ Ralph's Playlist Menu★.˚★🎧★˚.★
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Understand - Keshi

Enter choice:
```

Invalid Inputs:

```
★.˚★🎧★˚.★ Ralph's Playlist Menu★.˚★🎧★˚.★
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Understand - Keshi

Enter choice: Limbo - Keshi
Invalid input. Please enter a number (1-5):
```

Remove Song:

```
⋆.˚★🎧★˚.⋆ Ralph's Playlist Menu⋆.˚★🎧★˚.⋆
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Understand - Keshi

Enter choice: 2

Current Playlist
1. Understand - Keshi
2. Limbo - Understand
3. Angel - Keshi

Enter song number to remove: 3

Song deleted...

⋆.˚★🎧★˚.⋆ Ralph's Playlist Menu⋆.˚★🎧★˚.⋆
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Understand - Keshi
```

Next Song;

```
⋆.˚★🎧★˚.⋆ Ralph's Playlist Menu⋆.˚★🎧★˚.⋆
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Understand - Keshi

Enter choice: 3

⋆.˚★🎧★˚.⋆ Ralph's Playlist Menu⋆.˚★🎧★˚.⋆
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Limbo - Understand

Enter choice:
```

Play All Songs (Loop):

```
Enter choice: 4

Now Playing:
Understand - Keshi
Limbo - Understand

*.˚★🎧★˚.* Ralph's Playlist Menu*.˚★🎧★˚.*
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Limbo - Understand

Enter choice:
```

Quitting the program:

```
*.˚★🎧★˚.* Ralph's Playlist Menu*.˚★🎧★˚.*
1. Add Song
2. Remove Song
3. Next Song
4. Play All Songs
5. Quit
Currently Playing: Limbo - Understand

Enter choice: 5


EXITING...
```

Conclusion

After performing this activity, I have gained knowledge on how to construct and manipulate single- and double-linked lists. Initially, I had difficulty doing so since there were many pointers that needed to be handled, and each node had to be linked together correctly. Proper allocation and deletion of nodes were also something I had to keep in mind in order to prevent any issues regarding memory leaks and dangling pointers. I really had to think outside the box to be able to perform the ILOA so I could move on to the supplementary but after-hours of troubling shooting and finishing the last two functions (General Insert node and Delete node); I knew what I needed to do for the next steps.

The procedure portion of the activity helped me understand how double linked lists work by implementing different operations such as traversal, insertion, and deletion like what we did for the singly link list. And by applying those same concepts from the single list I managed to create the playlist where I can add, remove, play the next song, and display everything all the songs added to the playlist. The supplementary activity was quite a practice driven approach because I had to combine these operations into a menu-driven program, which made me think more about the logic of the program instead of just writing the code.

Overall, despite the amount of time spent trying to solve problems, troubleshooting and debugging I performed relatively well in carrying out this activity. My biggest area for improvement is becoming more comfortable with pointers and debugging logical errors. With more practice, I believe I will be able to understand linked lists better and write cleaner and more efficient programs without any help from google or any kind of assistance. I know I will probably need AI to make my life easier but I would like to have that talent and efficiency of making my own code from scratch.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Overview and QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization and rephrasing, code assistance (concepts and how it works)**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used only to provide guidance and explanations, while all coding, analysis, and final work were completed and understood by me. Grammar corrections and punctuations were corrected by AI.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.